

Hi,

In this document you'll find some tips on how to implement various parts announced in the homework.

Part 1 - API Gateway

Exercise 1

Scala/Java

1. First, create a new `KafkaProducer`:

```
private val KafkaProducerInstance = {  
  val producerProperties = new Properties;  
  producerProperties.setProperty("bootstrap.servers", "localhost:29092");  
  producerProperties.setProperty("key.serializer",  
    "org.apache.kafka.common.serialization.StringSerializer");  
  producerProperties.setProperty("value.serializer",  
    "org.apache.kafka.common.serialization.StringSerializer");  
  producerProperties.setProperty("linger.ms",  
    TimeUnit.SECONDS.toMillis(1L).toString)  
  producerProperties.setProperty("batch.size", "12KB")  
  new KafkaProducer[String, String](producerProperties)  
}
```

2. To send a new record, use this API:

```
KafkaProducerInstance.send(new ProducerRecord[String, String](  
  "the topic name", "the message key", "the payload you want to send"  
), "here you can add a callback to get the outcome of the record  
delivery")
```

In the end, flush to be sure to empty the buffer and get an outcome for all the records:

```
KafkaProducerInstance.flush()
```

3. The callback should implement the `org.apache.kafka.clients.producer.Callback` interface and the following method:

```
override def onComplete(metadata: RecordMetadata, exception:  
Exception): Unit = {
```

If the exception is null, it means the record was correctly delivered.

4. For the fallback storage, you can convert the input objects to JSON with the help of Jackson library:

```
val Instance = new ObjectMapper()
Instance.registerModule(DefaultScalaModule)
Instance.registerModule(new JavaTimeModule())

val eventJson = JsonMapper.Instance.writeValueAsString(visit)
```

Python

1. First, create a new Producer:

```
self.producer = Producer({'bootstrap.servers': 'localhost:29092',
                          'linger.ms': 1 * 1000, # 10 seconds
                          'batch.size': '12KB',
                          'retries': 3
                          })
```

2. To send a new record, convert it into a dictionary in order to extract the visitId and later use this API:

```
visit_dictionary = json.loads(visit_json)

self.producer.produce(
    topic="the topic name",
    value="the message key",
    key="the payload you want to send",
    on_delivery="here you can add a callback to get the outcome of the
record delivery"
)
```

In the end, flush to be sure to empty the buffer and get an outcome for all the records:

```
self.producer.flush(timeout=20)
```

3. The callback should implement the following method:

```
def handle_delivery_result(error, result: Message):
```

If the error is null, it means the record was correctly delivered.

4. For the fallback storage, you don't need to convert the records to JSON. They're already JSON strings in the input.

Part 2 - Batch layer

Exercise 1

The code is similar for Scala Spark and PySpark. For the sake of simplicity (less verbosity), I'll share here the snippets for Scala Spark only. If you find it disturbing, please let me know. If you're not alone, I'll add PySpark snippets as well.

1. Reading the Apache Kafka topic. The format is similar for PySpark and Scala Spark and consists of the following lines:

```
sparkSession.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:29092")
  .option("subscribe", "visits")
  .option("startingOffsets", "EARLIEST")
  .load()
```

2. To extract the event, you must read the *value* column and cast it into a string. Here is the API:

```
selectExpr("CAST(value AS STRING)")
```

3. Now it's time to extract the *eventTime* field. Since we only need this field in our processing, we can define a simple schema. In Apache Spark the schemas consists of a list of fields wrapped in a *StructType*:

```
StructType(Seq(
  StructField("eventTime", TimestampType)
))
```

And for the transformation:

```
// You can replace the "$" by functions.col(...) in PySpark
select($"value", functions.from_json($"value",
VisitTimestampOnlySchema).as("data"))
```

4. Define the checkpoint location and trigger:

```
.option("checkpointLocation", s"${prefixDir}/checkpoint")
.trigger(Trigger.ProcessingTime(3.seconds))
```

5. Use the foreachBatch sink. Inside, you can implement the following writing algorithm:

5.1. Classify an event as valid or invalid by adding a new column:

```
.withColumn("is_valid_event_time",
functions.when($"data.eventTime".isNull(), false).otherwise(true))
```

5.2. Calling the .cache() method on the DataFrame with the extra column.

5.3. Writing the valid records. You should filter on the is_valid_event_time column and add the partition columns by leveraging Apache Spark native capabilities:

```
.withColumn("year", functions.year($"data.eventTime"))
.withColumn("month", functions.month($"data.eventTime"))
.withColumn("day", functions.dayofmonth($"data.eventTime"))
.withColumn("hour", functions.hour($"data.eventTime"))
```

5.4. Writing data as Delta Lake table:

```
.write.format("delta").mode(SaveMode.Append)
.partitionBy("year", "month", "day",
"hour").save(s"${dataOutputDir}/valid")
```

The write will only succeed if you add the following lines to your SparkSession definition:

```
.config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
.config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
```

For PySpark, you must also add the Delta Lake as a dependency in the SparkSession:

```
.config('spark.jars.packages', 'io.delta:delta-core_2.12:2.2.0')
```

5.5 The logic for invalid data is the same except the part generating the partition columns because it must use the current time:

```
.withColumns(  
  Map(  
    "year" -> functions.year(functions.current_timestamp()),  
    "month" -> functions.month(functions.current_timestamp()),  
    "day" -> functions.dayofmonth(functions.current_timestamp()),  
    "hour" -> functions.hour(functions.current_timestamp()),  
  )  
)
```

Hope it helps 🙏
Bartosz.